

KIRA AI Incident Report

Cluster Summary

The cluster is a single-node `minikube` environment hosting a microservices-based application (including `auth`, `frontend`, `gateway`, `orders`, `product-service`, `user-service`, and a `boutique-postgres` database).

Currently, all **7 deployments** report status `1/1` available replicas, and all **8 pods** are in a `Running` state. However, telemetry reveals a coordinated restart event affecting 6 out of 8 pods, alongside critical anomalies in the `frontend` pod's resource consumption.

Key Findings

1. **Coordinated Cluster-Wide Restarts**:

Six pods (`auth`, `boutique-postgres-0`, `frontend`, `order-service`, `product-service`, and `user-service`) have restarted exactly **1 time**. This simultaneous restart across disparate services points to a node-level event rather than isolated application-level failures.

2. **Frontend Inactive/Idle State**:

The `frontend` pod is consuming an extremely low amount of CPU (`0.0000088` cores, or ~0.008 millicores). Compared to other microservices averaging `0.005`–`0.006` cores, the frontend is effectively idle. This suggests it is either not receiving traffic or is deadlocked/frozen post-restart.

3. **Single Point of Failure (SPOF)**:

All deployments are configured with `desired\_replicas: 1`. Any pod restart or node issue results in immediate service downtime, as there are no redundant replicas to absorb the impact.

Severity Assessment

Severity: SEV-2 (Major Incident - Degraded State)

Justification: Although all pods are currently reported as `Running`, the near-zero CPU usage on the `frontend` indicates a silent failure where user-facing traffic is likely blackholing. Furthermore, the coordinated restarts of 75% of the cluster's pods indicate recent node instability (OOM, Kubelet restart, or VM reboot) that has degraded cluster reliability.

Root Cause Analysis

Hypothesis 1: Node-Level Resource Exhaustion (OOM) or Node Reboot

Because `minikube` is a single-node local cluster, it is highly susceptible to CPU/Memory exhaustion. The fact that `gateway-6c7b7f78b5-bh7ck` and `orders-778f8fcf56-v7znz` did **not** restart, while the rest did, indicates either:

* **Kubelet Eviction / OOM Killer**: The Linux Out-Of-Memory (OOM) killer or Kubelet evicted multiple pods due to memory pressure on the node, restarting them sequentially.

* **Minikube Pause/Resume or VM Restart**: A local hypervisor pause/resume event or local docker daemon hiccup forced container runtimes to restart.

Hypothesis 2: Frontend Post-Restart Deadlock or Routing Issue

The `frontend` pod restarted and is now consuming almost zero CPU. This can happen if:

- * The frontend container completed its entrypoint initialization but is now stuck in an infinite sleep, deadlock, or waiting on a blocked network connection.
- * The `gateway` service is failing to route traffic to the `frontend` service (e.g., due to bad endpoint mapping after the restarts), resulting in zero incoming requests.

Recommended Remediation

1. **Investigate Last State of Restarted Pods**:

Inspect the `Last State` of the restarted pods to see if they terminated with `OOMKilled` (Exit Code 137) or a general error (Exit Code 1 or 255).

2. **Implement Multi-Replica Deployments**:

Increase replica counts for critical user-facing services (`frontend`, `gateway`, `auth`) to at least `2` to ensure high availability during single-pod recycles.

3. **Define Resource Requests and Limits**:

Ensure all deployments have explicit CPU/Memory requests and limits configured. This prevents a single runaway pod from triggering node-wide OOM cascades.

4. **Fix Frontend Traffic Flow**:

Check if the `gateway` is properly routing traffic to the `frontend`. Restart the `frontend` pod manually if it is deadlocked post-restart.

Suggested kubectl Commands

1. Diagnose Coordinated Restarts & Node Status

```
```bash
```

```
Check node resource utilization and uptime
```

```
kubectl top nodes
```

```
kubectl describe node minikube
```

```
Check recent cluster events for OOMKilling or eviction events
```

```
kubectl get events --sort-by='.metadata.creationTimestamp' -A
```

```
```
```

2. Inspect Restart Reasons for Pods

```
```bash
```

```
Check the termination reason (Exit Code/Reason) for the restarted pods
```

```
kubectl get pod -o jsonpath='{range .items[*]}{.metadata.name}{\n\tLast State:
```

```
"}{.status.containerStatuses[*].lastState}{\n"}{end}'
```

```
```
```

3. Troubleshoot the Frontend Anomaly

```
```bash
```

```
Check the logs of the frontend pod to see if it is stuck on startup
```

```
kubectl logs frontend-75bc59b865-td9rw --tail=100
```

```
Check previous container logs before the restart
```

```
kubectl logs frontend-75bc59b865-td9rw -p --tail=100
```

```
Confirm if endpoints are properly registered for the frontend service
```

```
kubectl get endpoints frontend
```

```
```
```

4. Apply Mitigations (Scale & Force Refresh)

```
```bash
Scale up critical services to prevent SPOF
kubectl scale deployment frontend --replicas=2
kubectl scale deployment auth --replicas=2

Safely rollout restart the frontend if it is in a hung/deadlocked state
kubectl rollout restart deployment frontend
```
```